

BORDER OPTICS — POORI RESEARCH SAMJHO

PDF 6 of 22 — Acquisition: Geocoding
`geocode_villages.py`, `geocode_villages_bhuvan.py`,
`merge_geocoded.py` — Poori Files, Line By Line

Sakshi ke liye, Sakshi ki apni research par — ab se asli code files khulna shuru. Har line, do baar explain: pehle uska research-purpose, fir uska Python-syntax. Koi line skip nahi, koi shortcut nahi.

Ye teen files milke woh 559-se-258 geocoding funnel banati hain jo PDF 5 mein dekha tha — yahan hum dekhenge *kaise*, ek-ek line padh ke: pehle OpenStreetMap se try karna (`geocode_villages.py`), fir jo miss hua use ISRO Bhuvan se dhundhna (`geocode_villages_bhuvan.py`), aur akhir mein sab kuch ek master table mein jodna (`merge_geocoded.py`). Development Log ki asli history bhi saath-saath weave hogi — har design-decision ke peeche ek real bug ya real fix chhupa hai.

Shuru Karte Hain — In Teen Files Ka Poora Role

PDF 5 mein humne dekha tha ki VVP-I ki official lists mein 559 villages the, aur BORDER OPTICS unme se sirf 258 ko successfully geocode kar paya — matlab, unka exact latitude/longitude nikaal paya, taaki satellite imagery unke liye query ho sake. Ab hum wahi funnel ke andar ghuskar dekhenge — kaunsi teen files, kis order mein, ye kaam karti hain.

Pehli file, `geocode_villages.py`, har gaon ko OpenStreetMap (Nominatim service ke through) mein dhundhti hai — ye pehla, primary attempt hai. Jo gaon isme miss ho jaate hain, unke liye doosri file, `geocode_villages_bhuvan.py`, ek alag source — ISRO ka Bhuvan portal — try karti hai, ek fallback ki tarah. Aakhir mein, teesri file, `merge_geocoded.py`, dono attempts se jo bhi successfully match hua, use ek single, clean master table mein jodti hai — jo aage har satellite-extraction aur analysis file use karti hai.

EXAMPLE — Do Dukaanon Mein Cheez Dhundhne Ki Analogy

Socho tu ek cheez kharidna chahti hai. Pehle tu apni sabse bhareyi, sabse trusted dukaan (Nominatim/OSM) mein jaati hai. Jo cheez wahan mil jaaye, wahan se le leti hai. Jo nahi milta, uske liye ek doosri, specialized dukaan (Bhuvan) try karti hai — jo shayad kuch aisi cheezein rakhti ho jo pehli dukaan mein nahi thi. Aakhir mein, tu dono dukaano se jo bhi mila, use ek hi bag (`merge_geocoded.py` ka master table) mein daal deti hai, taaki ghar jaake sab kuch ek jagah mil sake.

Ek zaroori baat shuru mein hi samajhna: ye teeno files 'v1' nahi hain — ye kayi rounds ke real bugs aur fixes ke baad ki, stable version hain. Jaise-jaise hum line-by-line padhenge, har jagah jahan koi unusual-lagta design-choice dikhega (jaise retry-loops, ya status-string ka poora set, ya dtype casting), wahan uske peeche ki asli history bhi batayenge — Development Log se, seedha.

Part A — geocode_villages.py: Gaon Ke Naam Ko Coordinates Mein Badalna

Is file ka kaam simple sunayi deta hai: har gaon ka naam lo, OpenStreetMap se uska latitude/longitude pata karo. Lekin asal mein ye kaafi tricky hai — network fail ho sakta hai, ek hi naam ke gaon do alag states mein ho sakte hain, aur kuch 'gaon' asal mein gaon hai hi nahi. Is file ka current version in sab problems ka fix hai, jo real experience se seekha gaya.

History — Ye File 'v4' Kyun Hai

File ke docstring mein hi likha hai: 'Village geocoding pipeline v4.' Iska matlab hai teen earlier versions the jo kaam nahi kiye — aur Development Log Entry 3 unki kahani batata hai. Sabse pehla version geopy naam ki ek popular Python library use kar raha tha Nominatim se baat karne ke liye — lekin wo baar-baar connection-timeout errors deta tha, chahe timeout value badha bhi di jaaye. Ye asal mein network ki problem nahi thi — ye khud geopy library ke andar ek quirk tha, ki wo apna timeout parameter sahi se resolve nahi kar rahi thi. Fix: geopy hataya, aur seedha requests library se Nominatim ke HTTP API ko call karna shuru kiya — jo file mein ab dikhega.

EXAMPLE — Ek Bharose-Wali Dukaan Vs Ek Chhoti, Seedhi Dukaan

Socho tu ek bade, complicated middleman (geopy) ke through order place kar rahi thi, aur wo baar-baar order confirm karne mein atak jaata tha — chahe usse jitna time do. Tu seedha dukaan (Nominatim ka apna HTTP API) se baat karne lagi, apne khud ke rules ke saath (kitni der wait karna, kitni baar retry karna) — aur problem gayab ho gayi. Kabhi-kabhi ek convenient-lagne waala shortcut (library) khud hi problem ban jaata hai.

Ek doosri, alag history bhi is file mein chhupi hai: 'Forest Block' entries ko geocoding se poori tarah exclude karna. Sikkim ki official VVP list mein kuch entries 'Forest Block' naam se hain — ye asal mein forest-survey compartments hain, inhabited villages nahi. Inhe geocode karne ki koshish karna time waste hai aur galat results de sakta hai, isliye ye poori tarah skip kiye jaate hain — `is_forest_block()` function mein ye check dikhega, aage.

Ab, Poori File — Line By Line

Docstring — File Ka Apna Introduction

```
"""Village geocoding pipeline v4: skips Forest Block entries, falls back to
a simplified query if the full query finds nothing, and distinguishes
real not-in-OSM misses from network failures so re-runs only retry the
failures.
"""
```

Purpose: Ye file ka apna, top-level introduction hai — koi bhi jo baad mein is file ko khole (khud Sakshi bhi, mahino baad), turant samajh jaaye is file ka current design kya hai aur kyun. **Syntax:** Triple-quotes (""...""") se bana ek multi-line string, jo file ke bilkul shuru mein hone ki wajah se Python ise *module docstring* maanta hai — ye code execute nahi hoti, sirf documentation hai.

Imports

```
import os
import time
import requests
import pandas as pd
```

Purpose: Char zaroori tools import ho rahe hain: os file-system operations ke liye (jaise folder banana), time wait/delay ke liye (Nominatim ko overload na karne ke liye), requests internet se data mangwane ke liye (Nominatim API call), aur pandas table/CSV operations ke liye. **Syntax:** import X poora module import karta hai, X naam se use hota hai. import X as Y same cheez, lekin chhote naam Y se refer karte hain — isliye aage pd.read_csv(...) dikhega, pandas.read_csv(...) nahi.

STATE_FILES — Kahan Se Kaunsa Data Aayega

```
STATE_FILES = {
    "Arunachal Pradesh": "data/raw/arunachal_pradesh_vvp_villages.csv",
    "Sikkim": "data/raw/sikkim_vvp_villages.csv",
    "Uttarakhand": "data/raw/uttarakhand_vvp_villages.csv",
    "Himachal Pradesh": "data/raw/himachal_pradesh_vvp_villages.csv",
}
```

Purpose: Har state ki raw (yaani abhi tak un-geocoded) VVP-I village list kahan padi hai, ye ek hi jagah define ho raha hai — taaki aage jo bhi loop chale, use hardcoded file-paths baar-baar likhne na pade. **Syntax:** Dictionary literal: {key: value, ...}. Har key ek state ka naam (string), value uski CSV file ka path (bhi string). Char states, char entries.

Nominatim Connection Settings

```
NOMINATIM_URL = "https://nominatim.openstreetmap.org/search"
HEADERS = {"User-Agent": "border_optics_research_sakshi_maske (contact: sakshimaske303@gmail.com)"}
```

Purpose: Pehli line Nominatim ke search API ka address hai. Doosri line ek 'User-Agent' header bhejti hai — ye Nominatim ki usage policy ka ek zaroori requirement hai: har request ke saath batana padta hai ki kaun request kar raha hai aur contact kaise kiya ja sakta hai, taaki agar koi problem ho (jaise research script bahut zyada requests bhej raha ho), Nominatim team contact kar sake, block karne se pehle. **Syntax:** Dono simple string aur dictionary assignments hain. HEADERS ek dictionary hai jisme sirf ek key-value pair hai.

Timing Aur Retry Constants

```
REQUEST_TIMEOUT = 20
MAX_RETRIES = 5
RETRY_WAIT_SECONDS = 8
MIN_DELAY_SECONDS = 1.5
```

Purpose: Ye char numbers poore script ke 'politeness aur patience' rules define karte hain: ek request ke liye zyada se zyada 20 second wait karo (REQUEST_TIMEOUT), agar fail ho toh 5 baar tak retry karo (MAX_RETRIES), har retry ke beech 8 second ruko (RETRY_WAIT_SECONDS), aur har successful request ke baad bhi kam se kam 1.5 second ruko agle se pehle (MIN_DELAY_SECONDS) — taaki Nominatim ke free, public server ko overload na karein. **Syntax:** Char alag-alag integer/float variables, sab module-level (file ke top par, kisi function ke andar nahi) — isliye poori file mein kahin bhi in naamo se access ho sakte hain.

Status Constants — Har Result Ka Ek Naam

```
STATUS_MATCHED = "matched"
STATUS_MATCHED_FALLBACK = "matched (fallback query, no block)"
STATUS_MATCHED_UNVERIFIED_STATE = "matched — WARNING: Nominatim result's state doesn't match expected state, verify manually"
STATUS_EXCLUDED_FOREST = "EXCLUDED — forest block, not an inhabited settlement"
STATUS_NO_MATCH_FINAL = "NO MATCH — not in OSM (tried full + simplified query)"
STATUS_FAILED_NETWORK = "FAILED AFTER RETRIES — network/timeout issue, retry later"
```

Purpose: Har gaon ka geocoding-attempt in cheh possible outcomes mein se kisi ek mein khatam hota hai, aur har outcome ka ek fixed, readable naam (string) hai — 'matched' se lekar 'network issue, retry later' tak. Ye constants isliye bane hain taaki poori file mein kahin bhi 'matched' jaisi string ko haath se type na karna pade (jahan ek typo bug bana sakta hai) — hamesha STATUS_MATCHED naam use hota hai. **Syntax:** Cheh string variables, sab UPPERCASE naam se (Python convention: UPPERCASE naam 'constant' ka signal deta hai — matlab ye value badlegi nahi, kam se kam is script ke chalte waqt).

is_forest_block() — Fake Villages Ko Pehchaanna

```
def is_forest_block(habitation):
    return "forest block" in str(habitation).lower()
```

Purpose: Ye function check karta hai ki kya ek gaon ka naam 'forest block' shabd contain karta hai — agar haan, toh ye asal mein koi inhabited village nahi hai, balki ek forest-survey compartment hai jo galti se VVP list mein 'village' ki tarah reh gaya (Sikkim list mein ye issue tha, jaisa upar history mein dekha). **Syntax:** def function_name(parameter): ek naya function define karta hai jo ek input (habitation) leta hai. str(habitation) use string mein convert karta hai (safety ke liye, agar kabhi number ya khaali value aa jaaye), .lower() sab lowercase kar deta hai (taaki 'Forest

Block' aur 'forest block' dono match ho), aur in check karta hai ki chhota string bade string ke andar hai ya nahi. return is check ka result (True ya False) wapas bhej deta hai.

EXAMPLE — BRTF Camps Aur Labour Camps — Ek Aur, Zyada Zaroori Kahani

Forest Blocks se bhi zyada interesting ek finding Development Log Entry 3 mein hai. Arunachal Pradesh ke jo villages geocode NAHI hue, unme se kayi asal mein civilian revenue villages hain hi nahi — ye Border Roads Task Force (BRTF) camps, army staging huts, labour camps, aur administrative headquarters hain, jaise 'Walong BRTF Camp', 'Misai Labour Camp', 'Staging Hut Krosam'. Ye VVP-I ki official 'priority village' list mein isliye hain kyunki scheme border ke saath har inhabited point ko target karti hai, chhota ho ya transient — lekin ye kisi civilian geospatial database (na OpenStreetMap, na Bhuvan) mein kabhi dikhenge hi nahi, kyunki inhe kabhi 'village' ki tarah classify hi nahi kiya gaya. Ye sirf ek data-gap nahi hai — ye khud is baat ka ek real, substantive proof hai ki ek securitized border par 'village-level development' ka matlab kya hota hai zameen par: army camps aur permanent civilian gaon ka mix, jisme se sirf ek hissa satellite study mein measure ho paata hai.

normalize_state() — State Ke Naam Ko Compare Karne Layak Banana

```
def _normalize_state(name):  
    return " ".join(str(name).lower().split())
```

Purpose: Ye chhota helper function state ke naam ko ek consistent, comparable form mein laata hai — taaki 'Arunachal Pradesh' (extra space ke saath) aur 'arunachal pradesh' (lowercase) dono same maane jaayein jab compare kiya jaaye. Syntax: `str(name).lower()` sab lowercase karta hai. `.split()` (bina kisi argument ke) string ko whitespace par todta hai aur extra spaces ignore karta hai, ek list words ki deta hai. `".join(...)"` un words ko wapas ek single space se jod deta hai. Result: consistent spacing, sab lowercase.

geocode_query() — Asli Nominatim API Call

```
def geocode_query(query, expected_state):  
    params = {"q": query, "format": "json", "limit": 5, "addressdetails": 1}  
    for attempt in range(1, MAX_RETRIES + 1):  
        try:  
            resp = requests.get(NOMINATIM_URL, params=params, headers=HEADERS,  
                                timeout=REQUEST_TIMEOUT)  
            resp.raise_for_status()  
            data = resp.json()  
            if not data:  
                return None, None, False, False, False
```

Purpose: Ye function ek search query bhejta hai Nominatim ko aur uska jawaab handle karta hai. Pehle, search-parameters set kiye jaate hain: query text, JSON format mein jawaab chahiye, zyada se zyada 5 candidates milein, aur address ka poora detail

(state, district waghera) bhi chahiye. Fir, MAX_RETRIES (5) baar tak koshish ka loop shuru hota hai. Har attempt mein: Nominatim ko GET request bhejo, agar server ne error diya toh turant exception uthao (raise_for_status()), jawaab ko JSON se Python data mein convert karo, aur agar bilkul kuch nahi mila (khaali result), turant ek 'genuinely no result' signal wapas bhejo — ye network problem nahi hai, ye sach mein OSM mein nahi mila. Syntax: for attempt in range(1, MAX_RETRIES + 1): ek loop jo 1 se 5 tak chalta hai (range(1, 6) 1,2,3,4,5 deta hai — upper limit exclusive hoti hai Python mein, isliye +1). try: block ke andar wo code hai jo fail ho sakta hai — agar fail hua, neech ke except use pakdega (aage dekhenge). return None, None, False, False, False ek saath paanch values wapas bhejta hai, comma se separated — Python mein ise 'tuple return' kehte hain.

```
expected_norm = _normalize_state(expected_state)
for candidate in data:
    candidate_state = candidate.get("address", {}).get("state", "")
    if _normalize_state(candidate_state) == expected_norm:
        return float(candidate["lat"]), float(candidate["lon"]), True, False, True
```

Purpose: Yahan asli 'Kharman-Haryana' waala safety-check ho raha hai (Part B mein Bhuvan ke liye bhi yahi tarah ka check dikhega — same lesson, do jagah apply). Jo bhi candidates Nominatim ne bheje (zyada se zyada 5), unme se ek-ek karke dekha jaata hai: kya iska state, humare expected state se match karta hai? Agar haan, turant iska latitude/longitude wapas bhejo, saath mein 'matched' aur 'state verified' ke True flags. Syntax: candidate.get("address", {}).get("state", "") ek safe, nested lookup hai — agar "address" key exist nahi karti, empty dictionary {} milta hai (crash nahi hota), aur agar uske andar "state" nahi hai, khaali string "" milta hai. .get(key, default) ye pattern poore file mein baar-baar dikhega — 'agar na mile, crash mat karo, default do.' float(...) string ko decimal number mein convert karta hai, kyunki Nominatim coordinates string ki tarah bhejta hai.

```
top = data[0]
return float(top["lat"]), float(top["lon"]), True, False, False
except requests.exceptions.RequestException as e:
    print(f" attempt {attempt}/{MAX_RETRIES} failed for '{query}': {e}")
    if attempt < MAX_RETRIES:
        time.sleep(RETRY_WAIT_SECONDS)
return None, None, False, True, False
```

Purpose: Agar koi bhi candidate ka state match nahi hua (loop upar wale block mein kisi ne 'return' nahi chalaya), toh top candidate (pehla, jo Nominatim ne sabse relevant maana) le liya jaata hai, lekin is baar 'state verified' False ke saath — ye ek warning-signal hai ki result use karne se pehle manually check kar lena chahiye. Agar poori request hi fail ho gayi (network error), except block chalta hai: error print karo, aur agar aur retries bache hain, thodi der ruko fir try karo. Agar loop poora khatam ho gaya bina kisi success ke, function bolta hai 'network failure ki wajah se poori tarah

fail hua.' Syntax: `except requests.exceptions.RequestException as e:` sirf network/HTTP se judi errors ko pakadta hai (na ki har tarah ki error — ye deliberately specific hai). `f"...{attempt}...{e}"` ek f-string hai jisme curly-braces ke andar variables seedhe text mein embed ho jaate hain. Function ke bilkul end mein wo return line loop ke *bahar* hai (indentation dekho — ye for ke andar nahi), isliye ye sirf tab chalti hai jab poora retry-loop bina success ke khatam ho jaaye.

build_query() — Search Text Banana

```
def build_query(row, state, use_block=True):
    parts = [str(row.get("Habitation", "")).strip()]
    if use_block and "Block" in row and pd.notna(row["Block"]) and "unresolved" not in
    str(row["Block"]).lower():
        parts.append(str(row["Block"]).strip())
    if "District" in row and pd.notna(row["District"]):
        parts.append(str(row["District"]).strip())
    parts.append(state)
    parts.append("India")
    return ", ".join([p for p in parts if p])
```

Purpose: Ye function ek CSV row se ek achhi, comma-separated search-string banata hai Nominatim ke liye — jaise 'Kaho, Anjaw, Arunachal Pradesh, India'. Shuru mein sirf gaon ka naam (Habitation) hota hai. Agar `use_block=True` hai (default) aur Block-naam available hai aur wo 'unresolved' nahi hai, toh Block bhi add hota hai — ye location ko aur specific banata hai. District bhi add hota hai agar available ho. Aakhir mein state aur 'India' hamesha add hote hain, aur sab kuch comma se jod diya jaata hai. Syntax: `use_block=True` ek *default parameter* hai — agar function call karte waqt ye value nahi di jaaye, True maana jaata hai. Poora if condition ek saath char cheezein check karta hai and se joda hua — sab True honi chahiye tabhi Block add hoga. `pd.notna(...)` pandas ka function hai jo check karta hai ki value 'missing' (NaN) toh nahi hai. Last line mein `[p for p in parts if p]` ek *list comprehension* hai — 'parts list ke har item p ko rakho, sirf agar p khaali nahi hai' — taaki khaali strings comma-list mein na aayein. `", ".join(...)` sabko comma-space se jod deta hai.

process_row() — Ek Gaon Ka Poora Attempt: Full Query, Fir Fallback

```
def process_row(row, state):
    habitation = row.get("Habitation", "")

    if is_forest_block(habitation):
        return None, None, STATUS_EXCLUDED_FOREST, build_query(row, state)

    query_full = build_query(row, state, use_block=True)
    lat, lon, ok, net_failed, state_verified = geocode_query(query_full, state)
    if ok:
        return lat, lon, (STATUS_MATCHED if state_verified else
        STATUS_MATCHED_UNVERIFIED_STATE), query_full
    if net_failed:
```

```
return None, None, STATUS_FAILED_NETWORK, query_full
```

Purpose: Ye function ek single gaon ke liye poora decision-flow chalata hai. Sabse pehle: kya ye forest block hai? Agar haan, seedha exclude karo, koi API call bhi nahi. Nahi toh, Block ke saath poori query banao aur try karo. Agar successful (ok True), toh return karo — lekin status ye decide karega ki state verify hua tha ya nahi (conditional expression se). Agar network hi fail hua, turant wo bhi report karo — is case mein simplified query try karne ka koi fayda nahi, kyunki problem query mein nahi, connection mein hai. **Syntax:** lat, lon, ok, net_failed, state_verified = geocode_query(...) — ye *tuple unpacking* hai: function ne paanch values ek saath return ki thi, yahan wo paanch alag variables mein automatically bant jaati hain, order ke hisaab se. STATUS_MATCHED if state_verified else STATUS_MATCHED_UNVERIFIED_STATE ek *conditional (ternary) expression* hai — 'agar state_verified True hai, pehla value lo, nahi toh doosra' — ek line mein if/else jaisa kaam.

```
time.sleep(MIN_DELAY_SECONDS)

query_simple = build_query(row, state, use_block=False)
if query_simple != query_full:
    lat, lon, ok, net_failed, state_verified = geocode_query(query_simple, state)
    if ok:
        return lat, lon, (STATUS_MATCHED_FALLBACK if state_verified else
STATUS_MATCHED_UNVERIFIED_STATE), query_simple
    if net_failed:
        return None, None, STATUS_FAILED_NETWORK, query_simple

return None, None, STATUS_NO_MATCH_FINAL, query_simple
```

Purpose: Agar poori query (Block ke saath) fail hui aur wo network-problem nahi thi (matlab, genuinely OSM mein Block-level detail ke saath nahi mila), toh ek chhoti si wait ke baad, ek *simplified* query try hoti hai — Block hataa ke, sirf gaon+district+state+India. Kabhi Block ka naam OSM mein match hi nahi hota (thoda alag spelling ho sakta hai), isliye usse hataana kabhi-kabhi match la deta hai. Ye bhi fail ho jaaye (aur network-issue na ho), toh final answer hai: 'sach mein OSM mein nahi hai, dono tareeke try kar liye.' **Syntax:** if query_simple != query_full: ye check zaroori hai kyunki agar row mein Block field hi nahi thi shuru se, toh query_simple aur query_full same hi honge — dobara wahi query bhejne ka koi matlab nahi, time aur API-calls dono waste honge.

geocode_state() — Ek Poore State Ke Liye Loop, Resume-Logic Ke Saath

```
def geocode_state(state, filepath):
    print(f"\n--- Geocoding {state} ---")
    df = pd.read_csv(filepath)
```

```
out_path = f"data/processed/{state.lower().replace(' ', '_')}_geocoded.csv"
os.makedirs("data/processed", exist_ok=True)
```

Purpose: Ye function ek poore state (jaise 'Arunachal Pradesh') ke saare villages process karta hai. Pehle, raw CSV padhi jaati hai. Phir output file ka naam banaya jaata hai — state ka naam lowercase, space ko underscore se replace (jaise 'arunachal_pradesh_geocoded.csv'). Aakhir mein, data/processed folder create kiya jaata hai agar pehle se na ho — taaki save karte waqt error na aaye. Syntax: `state.lower().replace(' ', '_')` — chained method calls: pehle sab lowercase, fir space ko underscore se replace. `os.makedirs(path, exist_ok=True)` ek folder banata hai; `exist_ok=True` ka matlab hai agar folder pehle se hai, error mat do, chup-chaap aage badho.

Resume Logic — Kaam Jahan Chhoda Tha Wahin Se Shuru

```
terminal_statuses = {
    STATUS_MATCHED, STATUS_MATCHED_FALLBACK,
    STATUS_MATCHED_UNVERIFIED_STATE,
    STATUS_EXCLUDED_FOREST, STATUS_NO_MATCH_FINAL,
    "matched via Bhuvan (district-verified)",
    "NO MATCH — Bhuvan also failed or district mismatch",
}
```

Purpose: Ye set define karta hai ki kaunse status 'final, dobara try karne ki zaroorat nahi' maane jaate hain. Interesting baat: isme `geocode_villages_bhuvan.py` ke status-strings bhi directly likhe hain ('matched via Bhuvan...', 'NO MATCH — Bhuvan...') — ye koi accident nahi hai, ye ek real bug ka fix hai (neeche history mein dekhenge). Syntax: Set literal: {item1, item2, ...}. Ek set mein order fix nahi hota aur duplicates khud-ba-khud remove ho jaate hain — yahan ye zaroori nahi ki order matter kare, sirf 'is value set mein hai ya nahi' check karna hai (jo sets ke liye bahut fast operation hai).

EXAMPLE — Ek Real Resume-Bug Ki Kahani

Development Log Entry 3 mein ek genuine bug record hai: pehle, jab `geocode_villages.py` dobara chalaya jaata tha (Bhuvan fallback pass ke baad), uska 'already done' check sirf apne khud ke status-strings ko pehchaanta tha — Bhuvan ke status-strings ('matched via Bhuvan...') ko nahi. Iska matlab: jo rows Bhuvan ne match ki thi, wo is script ko 'incomplete' dikhti thi, aur ye unhe *dobara* process karne ki koshish karta tha — temporarily Sikkim aur Uttarakhand ke matched counts ko wapas gira deta tha, jab tak ye pakda aur restore nahi kiya gaya. Fix: `terminal_statuses` set mein Bhuvan ke status-strings bhi explicitly add kar diye gaye, taaki ye script unhe 'already done, chhod do' samjhe. Ye ek achha reminder hai: jab do alag scripts ek hi file par kaam karte hain, unka 'ye kaam ho chuka hai' ka definition ek doosre se match hona chahiye — nahi toh ek script dusre ka kaam accidentally overwrite kar sakta hai.

```

if os.path.exists(out_path):
    existing = pd.read_csv(out_path)
    if len(existing) == len(df) and "geocode_status" in existing.columns:
        done_mask = existing["geocode_status"].isin(terminal_statuses)
        print(f" Found existing progress file: {done_mask.sum()}/{len(df)} rows already
finalized.")
        df = existing
    else:
        df["geocode_query_used"] = None
        df["latitude"] = None
        df["longitude"] = None
        df["geocode_status"] = None
    else:
        df["geocode_query_used"] = None
        df["latitude"] = None
        df["longitude"] = None
        df["geocode_status"] = None

```

Purpose: Ye poora block decide karta hai: kya ye ek fresh start hai, ya ek pehle-se-shuru-hui run ko continue karna hai? Agar output file already exist karti hai *aur* uski length original data se match karti hai *aur* usme progress-tracking column hai, toh purani file hi use ki jaati hai (jisme kuch rows already 'done' honge) — is script ka progress wahin se continue hoga. Nahi toh (fresh start, ya koi mismatch), naye, khaali tracking columns bana diye jaate hain — sab None (khaali) se shuru. Syntax: `os.path.exists(path)` check karta hai file hai ya nahi. `len(existing) == len(df)` compare karta hai row-count — agar mismatch hai, purani file trust nahi ki ja sakti (shayad source data hi badal gaya). `.isin(terminal_statuses)` har row ke liye True/False batata hai ki uska status set ke andar hai ya nahi — ye ek poori column par ek saath kaam karta hai (loop likhne ki zaroorat nahi). `.sum()` True values ginta hai (Python mein True=1, False=0 hota hai jab number ki tarah use ho). `df["col"] = None` se ek naya column ban jaata hai, sab rows mein khaali value ke saath.

Main Loop — Har Row Process Karna, Checkpoint Ke Saath

```

for i, row in df.iterrows():
    status = row.get("geocode_status")
    if pd.isna(status) and status in terminal_statuses:
        continue

    lat, lon, new_status, query_used = process_row(row, state)

    df.at[i, "geocode_query_used"] = query_used
    df.at[i, "latitude"] = lat
    df.at[i, "longitude"] = lon
    df.at[i, "geocode_status"] = new_status

    time.sleep(MIN_DELAY_SECONDS)

```

Purpose: Ye poore state ke saare gaon, ek-ek row karke process karta hai. Har row ke liye pehle check: kya ye pehle se 'terminal' (final, done) status rakhti hai? Agar haan, continue — is row ko skip karo, agli par jao. Nahi toh, process_row() call karo (jo humne upar dekha), aur uske results ko dataframe mein wahi row, wahi columns mein likh do. Har row ke baad ek chhota sa wait — taaki Nominatim server ko overload na karein. Syntax: df.iterrows() dataframe ki har row par loop chalata hai, har baar ek index (i) aur wo row (row) deta hai. continue ek loop-control keyword hai — turant is iteration ka baaki code skip karke agli iteration par chala jaata hai. df.at[i, "column"] = value ek specific row-column cell ko directly update karta hai — ye .loc se milta-julta hai lekin single-cell updates ke liye zyada fast hai.

```
if (i + 1) % 10 == 0:
    df.to_csv(out_path, index=False)
    matched_so_far = df["geocode_status"].isin([STATUS_MATCHED,
STATUS_MATCHED_FALLBACK]).sum()
    print(f" {i + 1}/{len(df)} processed... ({matched_so_far} matched so far)")

df.to_csv(out_path, index=False)
```

Purpose: Har 10 rows ke baad, progress file ko CSV mein save kar diya jaata hai — ye 'checkpoint saving' hai. Agar script kabhi beech mein crash ho jaaye (network down, laptop band, kuch bhi), zyada se zyada 10 rows ka kaam hi dobara karna padega, poora state nahi. Loop khatam hone ke baad, ek aakhri, final save bhi hoti hai — taaki sabse aakhri rows (jo shayad 10 ka multiple na hon) bhi zaroor save hon. Syntax: (i + 1) % 10 == 0 — % modulo operator hai, jo division ka remainder deta hai. Ye check True hota hai jab i+1 10, 20, 30... ho, matlab har 10-vi row ke baad. i+1 isliye (na sirf i) kyunki index 0 se shuru hota hai, lekin 'row number' ginti 1 se shuru karna zyada natural hai.

Final Summary — Kitna Match Hua

```
matched = df["geocode_status"].isin([STATUS_MATCHED,
STATUS_MATCHED_FALLBACK]).sum()
unverified = (df["geocode_status"] == STATUS_MATCHED_UNVERIFIED_STATE).sum()
excluded = (df["geocode_status"] == STATUS_EXCLUDED_FOREST).sum()
no_match = (df["geocode_status"] == STATUS_NO_MATCH_FINAL).sum()
failed = (df["geocode_status"] == STATUS_FAILED_NETWORK).sum()
print(f" Done: {matched} matched, {unverified} matched but state-unverified (check
these), "
      f"{excluded} excluded (forest block), {no_match} not in OSM, {failed} network-failed
(retry next run). "
      f"Saved to {out_path}")
return df
```

Purpose: State ka poora processing khatam hone ke baad, har outcome-category ki counting hoti hai — kitne matched, kitne unverified, kitne excluded, waghera — aur ek human-readable summary print hoti hai. Function aakhir mein poora, updated

dataframe wapas bhejta hai, taaki `__main__` block (aage dekhenge) use aage use kar sake. Syntax: Har line ek pattern repeat karti hai: `(condition).sum()` — pehle ek `True/False` column banao (comparison se), fir `True` values ginno. Aakhri `print(...)` ek f-string hai jo teen alag lines mein likha gaya hai — Python mein consecutive string literals (bina kisi operator ke beech mein) automatically jud jaate hain ek lambi string mein, jo lambe messages ko readable rakhne ka ek tareeka hai.

`__main__` Block — Script Chalne Par Kya Hota Hai

```
if __name__ == "__main__":
    all_results = {}
    for state, path in STATE_FILES.items():
        all_results[state] = geocode_state(state, path)

    print("\n=== SUMMARY ===")
    for state, df in all_results.items():
        matched = df["geocode_status"].isin([STATUS_MATCHED,
STATUS_MATCHED_FALLBACK]).sum()
        unverified = (df["geocode_status"] ==
STATUS_MATCHED_UNVERIFIED_STATE).sum()
        print(f'{state}: {matched}/{len(df)} geocoded, {unverified} state-unverified (manual
check recommended)')
```

Purpose: Jab ye file seedhe chalayai jaati hai (na ki kisi doosri file se import ki jaati hai), ye poora block chalta hai. Har state ke liye `geocode_state()` call hota hai, result ek dictionary mein store hota hai. Fir sabke liye ek combined, final summary print hoti hai — poore run ka overview, ek saath. Syntax: `if __name__ == "__main__":` ek Python idiom hai jo check karta hai 'kya ye file directly run ho rahi hai?' — agar koi doosri file isse import kare, ye block nahi chalega (sirf functions/constants available honge). `STATE_FILES.items()` dictionary ke saare key-value pairs par loop karta hai, ek saath dono deta hai.

```
print("\nRun the script again to retry any 'FAILED AFTER RETRIES' rows (network
issues).")
print("Rows marked 'NO MATCH — not in OSM' or 'EXCLUDED — forest block' are final
and won't be retried.")
print("Rows marked state-unverified matched SOMETHING in Nominatim, but no
candidate's returned "
      "state matched the expected state — worth a manual look before trusting the
coordinates.")
```

Purpose: File ke bilkul aakhir mein, teen helpful reminder-messages print hote hain — ye future Sakshi (ya koi bhi jo ye script chalaye) ko batate hain ki agle steps kya honge: fail hui rows ke liye script dobara chalo, 'no match'/'excluded' rows permanent hain, aur 'unverified' rows ko manually check karna chahiye trust karne se pehle. Syntax: Teeno seedhe `print()` calls hain, koi naya logic nahi — sirf user ko guide karne ke liye documentation-jaisa output.

Part B — geocode_villages_bhuvan.py: Jo OSM Se Chhoot Gaya, Usko Bhuvan Se Dhundhna

Part A ki file har gaon ko OpenStreetMap (Nominatim) se dhundhti hai. Jo miss ho jaate hain (status 'NO MATCH — not in OSM'), unke liye ek doosra, alag data-source try kiya jaata hai: Bhuvan — ISRO ka apna, Census-linked village geocoding portal. Ye file wahi fallback-pass implement karti hai.

History — Kyun Ye Doosri File Zaroori Thi, Aur Do Real Bugs

Bhuvan API Nominatim se richer data deta hai — village code, household count, population, coordinates ke saath. Lekin testing mein ek genuine, important limitation mili: Bhuvan state ya district ke hisaab se filter nahi karta. Ek common gaon-naam silently ek bilkul alag state ke same-naam gaon ko wapas bhej sakta hai.

EXAMPLE — Kharman Ka Kissa — Ek Real Bug Jo Isi File Ke Design Ko Justify Karta Hai

Development Log Entry 3 mein ek seedha, verifiable test record hai: 'Kharman' (Anjaw district, Arunachal Pradesh ka ek gaon) query karne par, Bhuvan ne Jhajjar district, Haryana ka ek same-naam gaon wapas bhej diya — bilkul galat jagah, hazaaron kilometer door. Agar ye check na hota, BORDER OPTICS ke dataset mein Arunachal Pradesh ka ek gaon, asal mein Haryana ke coordinates ke saath reh jaata — ek chhota sa naam-collision, ek bada data-integrity disaster ban sakta tha. Fix: ek validation step add kiya gaya jo sirf tabhi Bhuvan ka result accept karta hai jab uska returned district, expected district se match kare — mismatch ko ek genuine 'name collision' maana jaata hai, match nahi.

Do aur chhote, lekin real bugs bhi is phase mein mile. Pehla: pandas mein agar koi column poori tarah khaali (sab None) se shuru hoti hai, pandas use automatically float64 type maan leta hai — aur jab baad mein usme Bhuvan ke string-type village codes daalne ki koshish ki jaati hai, error aata hai (float column string accept nahi karta). Fix: aise columns ko explicitly object dtype mein cast karna, values daalne se pehle — ye file mein aage dikhega. Doosra: resume-logic ka wahi bug jo Part A mein dekha (Nominatim script Bhuvan ke matched rows ko dobara process karne ki koshish karta tha) — dono taraf se fix kiya gaya.

Ab, Poori File — Line By Line

Docstring

```
"""Bhuvan (ISRO) geocoding fallback for villages OSM missed; validates each
match against the expected district since Bhuvan doesn't filter by state.
Token expires ~1 day (BHUVAN_TOKEN env var).
"""
```

Purpose: File-level documentation — ye kya karti hai (OSM-miss villages ke liye fallback), kyun validation zaroori hai (Bhuvan state-filter nahi karta), aur ek practical operational detail (token roughly ek din mein expire ho jaata hai, isliye baar-baar refresh karna padta hai). **Syntax:** Triple-quoted module docstring, jaisa Part A mein dekha.

Imports Aur Environment Setup

```
import os
import time
import requests
import pandas as pd
from dotenv import load_dotenv

load_dotenv()
```

Purpose: Pehle char imports Part A jaise hi hain. Paanchwa naya hai: dotenv library se load_dotenv function, jo ek .env naam ki chhoti file se secret values (jaise API tokens) padh kar unhe environment variables ki tarah available banata hai — taaki secret tokens seedhe code mein hardcode na karne padein (jo GitHub par publicly commit ho jaate, agar kiya jaata). **Syntax:** from module import specific_thing — poora module import karne ke bajaye, sirf ek specific function/class import karna. load_dotenv() function ko call karna — bina parentheses ke ye sirf function ka reference hota, parentheses ke saath ye function ko turant chala deta hai.

BHUVAN_TOKEN — Credential Check

```
BHUVAN_TOKEN = os.environ.get("BHUVAN_TOKEN")
if not BHUVAN_TOKEN:
    raise RuntimeError(
        "BHUVAN_TOKEN environment variable is not set. Create a .env file "
        "(see .env.example) with BHUVAN_TOKEN=<your token>, or export it "
        "in your shell before running this script."
    )
```

Purpose: Environment se Bhuvan ka access-token nikala jaata hai. Agar wo missing hai (khaali ya None), script turant, samajh-mein-aane-wali error ke saath ruk jaata hai — na ki aage chalke, confusing 'authentication failed' error ke saath, requests ke through. Ye ek achhi practice hai: jitni jaldi ho sake fail karo, ek clear message ke saath. **Syntax:** os.environ.get("KEY") ek environment variable padhta hai, na milne par None deta hai (crash nahi karta). if not BHUVAN_TOKEN: True hota hai agar value None ho ya khaali string ho. raise RuntimeError(...) ek exception 'throw' karta hai — script ka execution turant rukk jaata hai, aur ye message terminal mein dikhta hai. Multi-line string yahan bhi consecutive-string-concatenation trick use karti hai.

STATE_FILES — Is Baar, Already-Geocoded Files

```
STATE_FILES = {
    "Arunachal Pradesh": "data/processed/arunachal_pradesh_geocoded.csv",
    "Sikkim": "data/processed/sikkim_geocoded.csv",
    "Uttarakhand": "data/processed/uttarakhand_geocoded.csv",
    "Himachal Pradesh": "data/processed/himachal_pradesh_geocoded.csv",
}
```

Purpose: Ye Part A ke STATE_FILES jaisa hi *dikhta* hai, lekin ek zaroori farak hai: ye data/raw/ ki raw village lists nahi hain, ye data/processed/ ki, Part A dwara *already geocoded* files hain — same files jo Part A ne geocode_state() ke andar banayi thi. Ye file un files ko phir se padhti hai, sirf unhi rows par focus karne ke liye jo 'NO MATCH' status ke saath reh gayi thin. **Syntax:** Same dictionary-literal pattern jaisa Part A mein.

Bhuvan Connection Settings

```
BHUVAN_URL =
"https://bhuvan-app1.nrsc.gov.in/api/api_proximity/curl_village_geocode.php"
REQUEST_TIMEOUT = 20
MAX_RETRIES = 3
RETRY_WAIT_SECONDS = 5
MIN_DELAY_SECONDS = 1.0
```

Purpose: Bhuvan API ka address, aur timing constants — Part A jaise concept, lekin thode alag numbers (kam retries, kam wait) kyunki Bhuvan ek alag service hai apni khud ki reliability characteristics ke saath. **Syntax:** Simple variable assignments, jaisa pehle dekha.

Status Constants Aur Retry-Target List

```
STATUS_MATCHED_BHUVAN = "matched via Bhuvan (district-verified)"
STATUS_BHUVAN_NO_MATCH = "NO MATCH — Bhuvan also failed or district mismatch"

RETRY_STATUSES = {
    "NO MATCH — not in OSM (tried full + simplified query)",
    "NO MATCH — Bhuvan also failed or district mismatch",
}
```

Purpose: Do naye status-strings define hote hain (Bhuvan ke apne outcomes ke liye), aur ek RETRY_STATUSES set — jo batata hai ki kaunsi rows is Bhuvan-pass mein try karni hain. **Interesting:** ye set khud Bhuvan ka apna 'no match' status bhi include karta hai — matlab, agar script dobara chalayi jaaye, wo rows jo pichli baar Bhuvan mein bhi match nahi hui thin, unhe bhi phir try kiya jaayega (shayad Bhuvan ka data update ho gaya ho tab tak). **Syntax:** Do string constants, fir ek set literal — dono patterns pehle dekh chuke hain.

district_matches() Aur state_matches() — Kharman-Jaisa Bug Rokna

```
def district_matches(returned_dist, expected_dist):
    if not returned_dist or not expected_dist:
```

```

    return False
    r = str(returned_dist).strip().lower()
    e = str(expected_dist).strip().lower()
    return r in e or e in r

```

Purpose: Ye function check karta hai ki Bhuvan ne jo district return kiya, wo humara expected district se milta-julta hai ya nahi. Agar dono mein se koi bhi missing hai, seedha False (match nahi maana jaata, safe side lete hain). Nahi toh, dono ko clean karke (lowercase, extra-space-hata-ke) compare kiya jaata hai — aur ye check 'ek dusre ke andar hai ya nahi' dono directions mein karta hai, kyunki kabhi ek string dusre ka substring hota hai (jaise 'Anjaw' aur 'Anjaw District'). Syntax: if not returned_dist or not expected_dist: — not khaali string ya None par True deta hai. r in e or e in r — substring check dono directions mein, or se joda hua.

```

def state_matches(returned_state, expected_state):
    # same fuzzy check as district_matches -- need this too or a same-named
    # district in a different state could sneak past
    if not returned_state or not expected_state:
        return False
    r = str(returned_state).strip().lower()
    e = str(expected_state).strip().lower()
    return r in e or e in r

```

Purpose: Bilkul district_matches() jaisa hi logic, lekin state ke liye. Comment khud explain karta hai kyun ye zaroori hai: sirf district match check karna kaafi nahi, kyunki do alag states mein same-named district ho sakta hai — dono checks saath mein chahiye poori tarah safe rehne ke liye. Syntax: Python mein # se shuru hone waali line ek comment hai — code execute nahi hoti, sirf insaan ke padhne ke liye. Baaki function structure upar wale jaisa hi hai.

bhuvan_lookup() — Asli API Call

```

def bhuvan_lookup(village_name):
    params = {"village": village_name, "token": BHUVAN_TOKEN}
    for attempt in range(1, MAX_RETRIES + 1):
        try:
            resp = requests.get(BHUVAN_URL, params=params, timeout=REQUEST_TIMEOUT)
            resp.raise_for_status()
            data = resp.json()
            if isinstance(data, list) and data:
                return data
            return [] # false / no result
        except requests.exceptions.RequestException as e:
            print(f" attempt {attempt}/{MAX_RETRIES} failed for '{village_name}': {e}")
            if attempt < MAX_RETRIES:
                time.sleep(RETRY_WAIT_SECONDS)
    return None # network failure after retries

```

Purpose: Ye Part A ke `geocode_query()` jaisa hi `retry-pattern` hai, lekin Bhuvan ke liye. Village-naam aur token bhejo, jawaab agar ek non-khaali list hai toh wapas bhejo. Agar Bhuvan ne 'kuch nahi mila' bola (jaise `false` ya khaali), ek khaali list wapas bhejo — is se aage code ko pata chalta hai 'genuinely nahi mila' vs 'network fail hua' (jo `None` deta hai, khaali list nahi). Syntax: `isinstance(data, list)` check karta hai ki data ka Python-type list hai ya nahi (Bhuvan kabhi-kabhi `false` bhi bhej sakta hai, jo list nahi hota). and data ye bhi check karta hai list khaali toh nahi. Teen alag return values — ek asli list (success), khaali list `[]` (genuinely no result), ya `None` (network fail) — ye teeno alag maane jaate hain aage ke code mein.

process_state() — Dtype Bug Ka Fix, Live

```
def process_state(state, filepath):
    print(f"\n--- Bhuvan fallback pass: {state} ---")
    df = pd.read_csv(filepath)

    if "bhuvan_vid" not in df.columns:
        df["bhuvan_vid"] = pd.Series([None] * len(df), dtype="object")
    else:
        df["bhuvan_vid"] = df["bhuvan_vid"].astype(object)

    if "bhuvan_population" not in df.columns:
        df["bhuvan_population"] = pd.Series([None] * len(df), dtype="object")
    else:
        df["bhuvan_population"] = df["bhuvan_population"].astype(object)
```

Purpose: Yahi hai wo float64-vs-string dtype bug ka fix, jo history section mein mention kiya tha. Do naye columns chahiye — `bhuvan_vid` (village ID, jo asal mein ek string hai) aur `bhuvan_population`. Agar ye columns already exist nahi karte, khaali values ke saath banaye jaate hain, lekin explicitly `dtype="object"` ke saath — na ki pandas ko guess karne dena (jo galat guess karta, float64). Agar already exist karte hain (kisi purani run se), unhe explicitly `.astype(object)` se convert kar diya jaata hai, safety ke liye. Syntax: `pd.Series([None] * len(df), dtype="object")` — `[None] * len(df)` ek list banata hai jisme `None` utni baar repeat hota hai jitni df mein rows hain (list multiplication trick). `pd.Series(..., dtype="object")` explicitly batata hai pandas ko ki ye column 'object' type ka hai — matlab, koi bhi Python type (string, number, `None`, kuch bhi) store kar sakta hai, na ki sirf numbers. `.astype(object)` ek existing column ko is flexible type mein convert karta hai.

```
retried, newly_matched = 0, 0

for i, row in df.iterrows():
    status = row.get("geocode_status")
    if status not in RETRY_STATUSES:
        continue

    retried += 1
```

```
habitation = str(row.get("Habitation", "")).strip()
expected_district = row.get("District", "")
```

```
results = bhuvan_lookup(habitation)
time.sleep(MIN_DELAY_SECONDS)
```

Purpose: Do counters shuru mein zero par set hote hain — kitni rows retry hui, kitni newly match hui, track karne ke liye. Loop har row check karta hai: kya iska status `RETRY_STATUSES` mein hai? Agar nahi (matlab ye row already matched thi, kisi aur tarah se), skip karo. Nahi toh, retried-counter badhao, gaon ka naam aur expected district nikalo, Bhuvan se lookup karo, aur thoda ruko. Syntax: `retried, newly_matched = 0, 0` — ek saath do variables ko values assign karna, tuple-unpacking ka ek chhota use. `retried += 1` ka matlab `retried = retried + 1` — shorthand increment.

```
if results is None:
    # network failure — leave status as-is, retryable on a future run
    continue

accepted = None
for candidate in results:
    if district_matches(candidate.get("dist_name"), expected_district) and \
       state_matches(candidate.get("state_name"), state):
        accepted = candidate
        break
```

Purpose: Agar Bhuvan se network-failure hua (`None`), is row ko chhod do — status waisa hi rehne do taaki future run phir try kare. Nahi toh, Bhuvan ne jo bhi candidates bheje unme se ek-ek karke dekho: kya iska district *aur* state dono match karte hain expected values se? Pehla jo match kare, use accept karlo aur loop rok do — aage aur candidates dekhne ki zaroorat nahi. Syntax: `\` (backslash) line ke end mein ek line-continuation character hai — Python ko batata hai ki ye logical line agli physical line mein continue hoti hai (lamba condition padhne mein aasaan banane ke liye). `break` loop ko turant rok deta hai, jaise hi pehla acceptable candidate mil jaaye.

```
if accepted:
    df.at[i, "latitude"] = float(accepted["latitude"])
    df.at[i, "longitude"] = float(accepted["longitude"])
    df.at[i, "geocode_status"] = STATUS_MATCHED_BHUVAN
    df.at[i, "bhuvan_vid"] = accepted.get("vid")
    df.at[i, "bhuvan_population"] = accepted.get("tot_p")
    newly_matched += 1
    print(f" MATCHED: {habitation} -> {accepted['dist_name']},
{accepted['state_name']}")
else:
    df.at[i, "geocode_status"] = STATUS_BHUVAN_NO_MATCH
if results:
    found_in = ", ".join(f"{c.get('dist_name')}/{c.get('state_name')}" for c in results)
```

```

        print(f" DISTRICT/STATE MISMATCH (discarded): {habitation} — Bhuvan only
found it in: {found_in}")
    else:
        print(f" NOT FOUND: {habitation}")

```

Purpose: Agar koi candidate accept hua, uske coordinates, status, aur extra Bhuvan-specific data (village ID, population) dataframe mein likhe jaate hain, aur success-message print hoti hai. Agar koi accept nahi hua — ya toh Bhuvan ne kuch bheja hi nahi (genuinely not found), ya usne kuch bheja lekin district/state match nahi hua (Kharman-jaisa case, discard kiya gaya) — dono situations mein different, informative messages print hoti hain, taaki jo bhi script chalaye, samajh sake *kyun* ye row match nahi hui. Syntax: `accepted.get("vid")` — dictionary se ek value safely nikalna (na milne par None, crash nahi). Ternary-jaisi if results: check karta hai list khaali toh nahi. `f"{c.get('dist_name')}/{c.get('state_name')}"` for c in results ek generator expression hai andar ", ".join(...) ke — har rejected candidate ka district/state list bana ke ek comma-separated string mein dikha deta hai, taaki full context mile.

```

df.to_csv(filepath, index=False)
print(f" Retried {retried} villages, newly matched {newly_matched}. Updated {filepath}")

```

Purpose: Poore state ka processing khatam hone ke baad, updated dataframe wapas *usi* file mein save kar diya jaata hai jahan se padha gaya tha (Part A ki output file — is baar directly overwrite, kyunki ye already-geocoded data ko update kar rahi hai, naya file nahi bana rahi), aur ek summary print hoti hai. Syntax: `df.to_csv(filepath, index=False)` — same file-path jo function ke shuru mein padha gaya tha, isliye ye effectively 'in-place update' hai.

__main__ Block

```

if __name__ == "__main__":
    for state, path in STATE_FILES.items():
        process_state(state, path)

    print("\n=== FINAL SUMMARY (after Bhuvan fallback) ===")
    matched_statuses = {"matched", "matched (fallback query, no block)",
STATUS_MATCHED_BHUVAN}
    for state, path in STATE_FILES.items():
        df = pd.read_csv(path)
        matched = df["geocode_status"].isin(matched_statuses).sum()
        print(f"{state}: {matched}/{len(df)} geocoded")

```

Purpose: Har state ke liye `process_state()` chalta hai (jo file ko in-place update kar deta hai). Fir, saari files dobara padhi jaati hain (fresh, updated state mein) aur ek final, combined summary print hoti hai — ab teeno tarah ke 'matched' status (plain Nominatim match, fallback-query match, ya Bhuvan match) sab ek saath ginte hain,

taaki poore pipeline ka true, final geocoding-success-rate dikhe. Syntax: Note ek subtlety: `matched_statuses` yahan sirf plain string literals `"matched"` aur `"matched (fallback query, no block)"` use karta hai, na ki Part A ke `STATUS_MATCHED/STATUS_MATCHED_FALLBACK` constants — kyunki ye ek alag file hai aur wo constants yahan import nahi kiye gaye. Values same hain, lekin isse ye dikhta hai ki duplicate string-literals ka risk hamesha thoda rehta hai jab constants share nahi kiye jaate — chhoti si inconsistency, lekin values manually match karte hain isliye kaam kar jaata hai.

Part C — merge_geocoded.py: Sab Kuch Ek Master Table Mein Jodna

Ye file PDF 2 mein bhi dekhi thi, ek supplementary, Python-foundations example ki tarah. Lekin humari rule saaf hai: har research code-file, jahan bhi use ho, poori, line-by-line, purpose+syntax dono ke saath dobara padhni hai — koi shortcut 'PDF 2 mein cover ho chuka' nahi chalega. Toh yahan bhi, poori file, ek baar aur, iss baar poore geocoding-pipeline ke context ke saath.

Ye teesri aur aakhri file, Part A aur Part B ke saare results ko — jo abhi tak char alag state-files mein bikhre hain — ek single, clean, 258-village master table mein jodti hai. Yahi master table hai jo aage har satellite-extraction script use karega.

Docstring

```
"""
BORDER OPTICS — Merge all geocoded villages into one master table
for upload to Google Earth Engine.
"""
```

Purpose: Batata hai file ka final goal — sirf 'jodna' nahi, balki specifically Google Earth Engine mein upload karne layak ek clean format banana (jo hum PDF 8 mein dekhenge, jab satellite extraction files khulengi). Syntax: Standard module docstring.

Import

```
import pandas as pd
```

Purpose: Sirf ek import chahiye — ye file koi network call nahi karti, sirf existing CSVs padhti aur jodti hai, isliye requests ki zaroorat nahi. Syntax: import X as Y pattern, jaisa pehle dekha.

FILES — Kahan Se Geocoded Data Aayega

```
FILES = {
    "Arunachal Pradesh": "data/processed/arunachal_pradesh_geocoded.csv",
    "Sikkim": "data/processed/sikkim_geocoded.csv",
    "Uttarakhand": "data/processed/uttarakhand_geocoded.csv",
    "Himachal Pradesh": "data/processed/himachal_pradesh_geocoded.csv",
}
```

Purpose: Yahi char files jo Part A ne banayi aur Part B ne update ki — ab final merge ke liye padhi jaayengi. Syntax: Dictionary literal, bilkul Part A/B ke STATE_FILES jaisa.

MATCHED_STATUSES — Sirf Successful Rows Chahiye

```
MATCHED_STATUSES = {
    "matched",
    "matched (fallback query, no block)",
}
```

```
"matched via Bhuvan (district-verified)",  
}
```

Purpose: Teen tarah ke status jo 'genuinely successfully geocoded' maane jaate hain — plain Nominatim match, fallback-query match, ya Bhuvan match. Ek zaroori baat notice karo: `STATUS_MATCHED_UNVERIFIED_STATE` (wo 'WARNING' waala status, jahan koi candidate ka state match nahi hua tha) is set mein nahi hai — matlab, wo rows is final master table mein *shaamil nahi* honghi, chahe unke paas coordinates ho bhi. Ye ek conservative, safe choice hai: agar state verify nahi ho paaya, use core sample mein na lo, jab tak koi manually check na kare. **Syntax:** Set literal, teen string members ke saath.

Main Merge Logic

```
rows = []  
for state, path in FILES.items():  
    df = pd.read_csv(path)  
    matched = df[df["geocode_status"].isin(MATCHED_STATUSES)].copy()  
    matched["state"] = state  
    # Himachal is just the small illustrative case study, not core sample  
    matched["is_core_sample"] = state != "Himachal Pradesh"  
    rows.append(matched[["Habitation", "District", "Block", "state", "latitude", "longitude",  
                        "is_core_sample"]])
```

Purpose: Ek khaali list se shuru, jisme har state ka processed data jama hoga. Har state ke liye: uski geocoded CSV padho, sirf successfully-matched rows rakho (baaki discard), ek naya 'state' column add karo (taaki baad mein pata chale ye row kis state se hai — original row mein ye nahi tha), aur ek 'is_core_sample' flag add karo jo batata hai ye row teen core states (jinpar H1/H3/H4 chalte hain) ka hissa hai ya sirf Himachal (illustrative). Aakhir mein, sirf zaroori columns chunke, is state ka processed data rows list mein add kar do. **Syntax:** `rows = []` khaali list. `df[condition].copy()` — boolean filtering se sirf matching rows chuno, aur `.copy()` se ek independent copy banao (pandas ki 'chained assignment warning' se bachne ke liye, jo warna aa sakti thi agar hum seedhe isi filtered dataframe ko aage modify karte). `matched["state"] = state` — ek nayi column bana rahe hain, har row mein wahi single value (current loop ka state-naam) bhar jaati hai. `state != "Himachal Pradesh"` ek comparison hai jo True/False deta hai — is boolean value ko seedha column mein daal diya, ye khud hi ek valid column ban jaata hai. `rows.append(...)` list ke end mein ek naya item (ye poora filtered dataframe) jod deta hai.

Sab States Ko Ek Saath Jodna

```
master = pd.concat(rows, ignore_index=True)  
master = master.rename(columns={"Habitation": "village", "District": "district", "Block":  
                               "block"})
```

Purpose: Ab jab rows list mein sab char states ka data alag-alag dataframes ki tarah jama ho chuka, unhe ek hi, badi dataframe mein jod diya jaata hai. Fir kuch column-naam clean kiye jaate hain — 'Habitation' ko 'village', 'District' ko 'district', 'Block' ko 'block' — lowercase, simpler naam, jo aage ki files mein consistently use honge. Syntax: `pd.concat(list_of_dataframes, ignore_index=True)` — sab dataframes ko rows ki tarah stack karta hai (upar-neeche jodta hai, saath mein nahi). `ignore_index=True` purane, alag-alag row-numbers ko bhula kar 0 se naya, continuous index banata hai. `.rename(columns={old: new, ...})` ek dictionary leta hai jahan key purana naam hai, value naya naam.

Har Gaon Ko Ek Unique ID Dena

```
master["village_id"] = range(1, len(master) + 1)
master = master[["village_id", "village", "district", "block", "state", "is_core_sample",
"latitude", "longitude"]]
```

Purpose: Har gaon ko ek sequential, unique number diya jaata hai (1, 2, 3, ... jitni bhi rows hain). Ye ID aage har analysis file mein is gaon ko refer karne ka sabse reliable tareeka banega (naam kabhi duplicate ho sakta hai, ID kabhi nahi). Fir, columns ko ek clean, consistent, logical order mein rearrange kiya jaata hai. Syntax: `range(1, len(master) + 1)` ek sequence banata hai 1 se lekar total-rows tak (jaisa Part A mein `range(1, MAX_RETRIES + 1)` dekha tha, wahi pattern). Ise seedha ek naye column mein assign karna, pandas automatically har row ko sequence ka agla number de deta hai. `master[[col1, col2, ...]]` — double square brackets, kyunki hum ek *list* of column-names pass kar rahe hain, jo dataframe ko un columns ko wahi order mein select-aur-rearrange karne ko kehta hai.

Save Aur Confirmation

```
master.to_csv("data/processed/border_optics_master_villages.csv", index=False)
print(f"Merged {len(master)} villages into
data/processed/border_optics_master_villages.csv")
print(master["state"].value_counts())
```

Purpose: Final master table CSV mein save hoti hai — ye wahi file hai jo aage border-distance computation, satellite extraction, aur har analysis script use karega. Fir do confirmation messages print hoti hain: kitne total villages merge hue, aur state-wise breakdown, taaki turant dikh jaaye numbers sahi hain ya nahi (258 total, 186/31/34/7 state-wise — jaisa PDF 5 mein dekha tha). Syntax: `index=False` — pandas ka apna internal row-number CSV mein nahi likhta (hume wo nahi chahiye, humare paas apna `village_id` hai). `.value_counts()` ek pandas method hai jo ek column ki har unique value kitni baar aayi, wo count karke dikhata hai — yahan 'state' column par, matlab har state ke kitne villages final table mein hain.

Teeno Files Ek Saath — Poora Funnel, Ab Code Ki Nazar Se

Ab jab teeno files line-by-line padh li, aayo ek baar poora funnel dobara dekhte hain — is baar PDF 5 ke sirf numbers ki tarah nahi, balki 'ye numbers kaise bane' ki tarah.

Step	File	Kya Hota Hai
1	geocode_villages.py	559 villages, Nominatim (OSM) se try — forest blocks skip, full query fir simplified query fallback
2	geocode_villages_bhuvan.py	Jo Step 1 mein miss hue, unhe Bhuvan se try — district+state verify karke
3	merge_geocoded.py	Dono steps ke successful matches ko ek 258-village master table mein jodna

Har jagah jahan ek design-choice thodi unusual lagti hai — retry-loops, status-constants ka poora set, dtype-casting, ya resume-logic — uske peeche ek real bug tha jo pehle actually hua tha: geopy ka timeout-quirk, Kharman-Haryana ka galat-state match, pandas ka float64-vs-string dtype clash, aur do alag scripts ke resume-logic ka ek dusre se disagree karna. Ye code 'perfect design' se nahi bana — ye iteration se bana, har bug ko genuinely fix karke.

EXAMPLE — Ek Zaroori Reminder — Yahi Hai 'Good Code' Ka Matlab

Naye coders (aur naye researchers) kabhi sochte hain ki 'achha code' matlab pehli baar mein hi perfect likh dena. Asal mein, jaisa in teeno files mein dikha, achha code aksar iterative hota hai — v1 mein ek problem thi, v2 ne use fix kiya lekin ek naya edge-case expose kiya, v3 ne wo fix kiya, v4 mein sab stable hua. Har retry-loop, har validation-check, har dtype-cast — ye sab 'scars' hain, real problems ke real fixes, na ki kisi textbook se copy ki hui best-practice. Jab tu khud code likhegi, yahi expect karna — pehli try mein perfect hona zaroori nahi, important ye hai ki jab bug mile, use samajh ke fix karna.

Is PDF Ka Recap — Ab Tak Kya Seekha

→ `geocode_villages.py`: 559 villages ko OpenStreetMap (Nominatim) se try karta hai — pehle poori query (Block ke saath), fail hone par simplified query, network-fail aur genuine-no-match ko alag track karta hai.

→ Forest Block entries (Sikkim list mein) exclude hoti hain — ye inhabited villages nahi, forest-survey compartments hain.

→ Har candidate ka state verify hota hai expected-state se — ye exact wahi safety-check hai jo Kharman/Haryana-jaisi galtiyon ko rokta hai.

→ Checkpoint saving (har 10 rows) aur resume-logic (`terminal_statuses` set) ka matlab: script kabhi crash ho, zyada se zyada 10 rows ka kaam dobara karna padta hai.

→ `geocode_villages_bhuvan.py`: jo Nominatim se miss hua, use ISRO ke Bhuvan portal se try karta hai — `district_matches()` aur `state_matches()` se Bhuvan ke 'state-filter nahi karta' waale gap ko cover karta hai.

→ Dtype-casting (`.astype(object)`) ek real pandas bug ka fix hai — khaali column float64 ban jaata hai, string data reject karta hai, jab tak explicitly object type na di jaaye.

→ `merge_geocoded.py`: dono sources ke successful matches (`MATCHED_STATUSES`) ko ek 258-village master table mein jodta hai — 'unverified state' wali rows conservatively exclude hoti hain.

→ Har retry-loop, validation-check, aur dtype-fix ke peeche ek real, documented bug hai — achha code perfect pehli-try se nahi, honest iteration se banta hai.

Aage kya: PDF 7 mein hum `select_control_villages.py` kholenge — akela, sabse complex file, jo 721 non-VVP control villages banati hai (jo H4 DiD test ke liye zaroori hain, PDF 4 mein dekha tha). Isi file mein wo poora, multi-stage contamination-fix ki history bhi code-level par dikhegi — 735 se 732 se 721 tak, jo PDF 4 mein numbers ki tarah dekha tha, ab hum dekhenge *kaise*, line by line.

Jab ready ho agle PDF ke liye, bas bol dena.